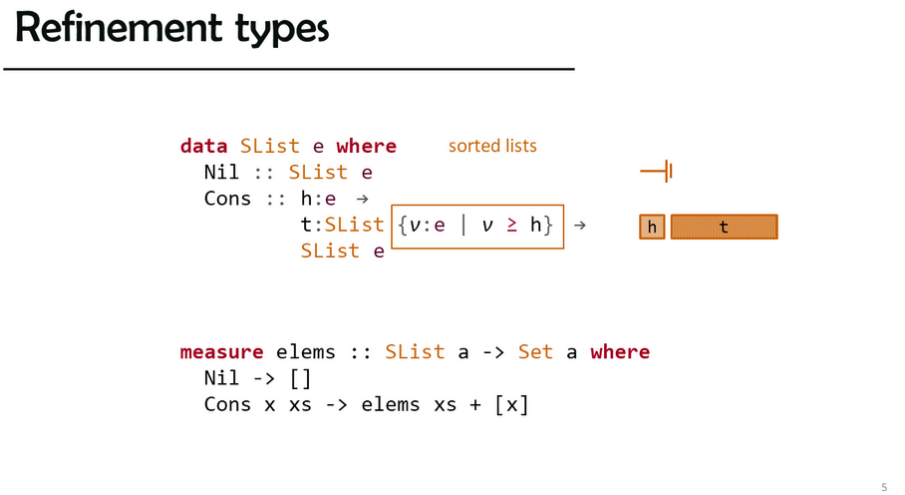
LECTURE 22

Synthesis with Refinement Types

(Developed in collaboration with Nadia Polikarpova from materials from her course)



In the previous lecture, we saw how refinement types could allow us to specify complex properties of a program, and how they can allow us to catch errors early, even when the program has not been completed. In this section, we now describe how these properties can be leveraged by a program synthesizer to efficiently generate a program that satisfies the requirements stated in the type signature. This lecture is based entirely on the work of Polikarpova, Kuraj and Solar-Lezama [1].

As a running example, we will continue to use the definition of `SList` from the slide in the previous lecture. In this definition, a sorted list `SList` can be constructed either as `Nil`, or as a `Const h t` of a head element `h` and a sorted list `t` of elements greater than `h`. Note that like functions, constructors also have named arguments, and the type of each argument can refer to the values of previous arguments.

We will also be using the recursive function `elems`, which is defined as a measure, allowing us to use it the type signature for functions involving `SLists`.

22.1 Round-Trip type checking

Bidirectional type checking

```
x:e  →  xs:SList e  →
  {v:SList e | elems v = elems xs ∪ {x}}
```



```
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

8

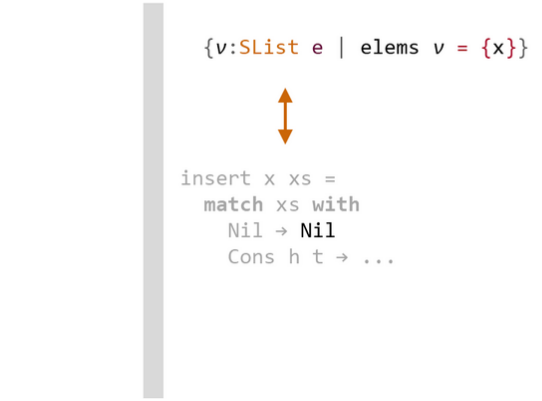
Bidirectional type checking

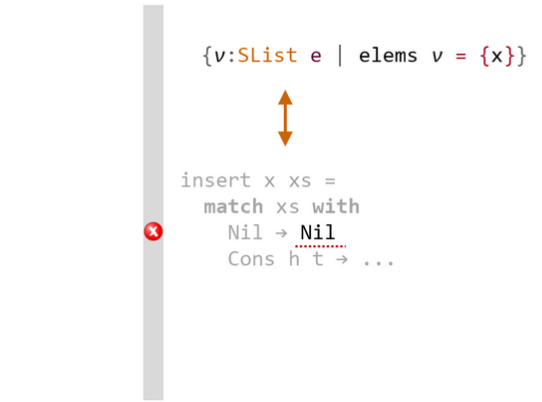
```
  {v:SList e | elems v = elems xs ∪ {x}}
```



```
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

9





One of the things that makes types very powerful for synthesis is the ability to reject incomplete programs. One particular formalism that is well established in the literature and that has proven very effective in program synthesis is *Bidirectional typechecking*. This is a type checking approach where we propagate information both top down (propagating information from the expected type of a function to the types of its constituent parts), as well as bottom up (propagating the known type of some components to get the type of their composition).

The figure shows an example of top-down propagation of the type signature of the function, to the expected type of the `match` expression, to the expected type of the `Nil` expression. Note that the `Nil` expression happens in a context where we know `xs` is empty, so $elems\ xs \cup \{x\} = \{x\}$. At this point,

the type checker can see there is a type mismatch between the desired type propagated top-down and the type of `Nil` propagated bottom-up.

Round-trip type checking is a generalization of bidirectional type checking. A key feature in round-trip type checking is that the type of a term can be progressively refined as information propagates up and down the expression tree.

Round-trip type checking

```
x:e  →  xs:SList e  →
  {v:SList e | elems v = elems xs ∪ {x}}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```

12

Round-trip type checking

```
  SList e
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)    h  (insert x ...)
```

14

Round-trip type checking

```
SList {v:e | v ≥ h}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```

h (insert x ...)

15

Round-trip type checking

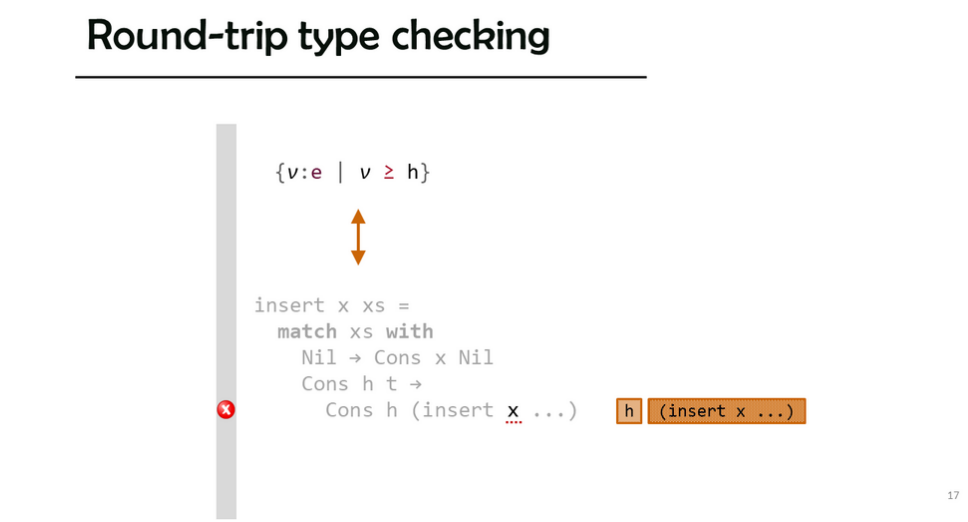
```
{v:e | v ≥ h}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```

h (insert x ...)

16



At a high-level, when bidirectional typechecking encounters a term `f x y` in a place where it expects a type `T`, it will first infer the type of `f` bottom up, then it will propagate that type top down to required types for `x` and `y`, and then based on the types of `x` and `y`, it will compute a type for `f x y` that it will compare with `T`. This process, however, requires the entire expression to be present before we can compare its type against the expected type `T`.

By contrast, in round-trip type checking, the type of `f` would immediately be refined based on our knowledge that it will eventually have to be matched against `T`. This allows information about the constraints from `T` to be propagated to `x` before we even know what the term `y` looks like.

To illustrate this idea, consider the example in the figure. First, the overall type of the function is propagated top down to the type of the `Cons` constructor. Ignoring for now the `elems` part of the type, we can see that we expect this constructor to produce an `SList`. The fact that the first argument is `h`, then needs to be propagated up to the type signature of `Cons`, which then uses this information to propagate back down a type for `insert` that depends on `h`.

At this point, a traditional bidirectional approach would have to compute a type for the `insert` term to match it against the expected type `SList {e | _v >= h}`. However, this would require completing the insert term before we can get its type. In contrast, round-trip type checking can propagate the expected type inside the function application to restrict the type expected of `x`. At this point, we can see that the required type for this parameter is $\{v : e | v \geq h\}$, which cannot be made to match the type of `x`, because `x` is unconstrained. So whereas traditional bidirectional typechecking would have had to check all terms of the form `insert x ...` one by one to reject `x` as the first parameter, round-trip typechecking was able to immediately reject the partially constructed term.

The details of the round-trip type checking algorithm can be found in the

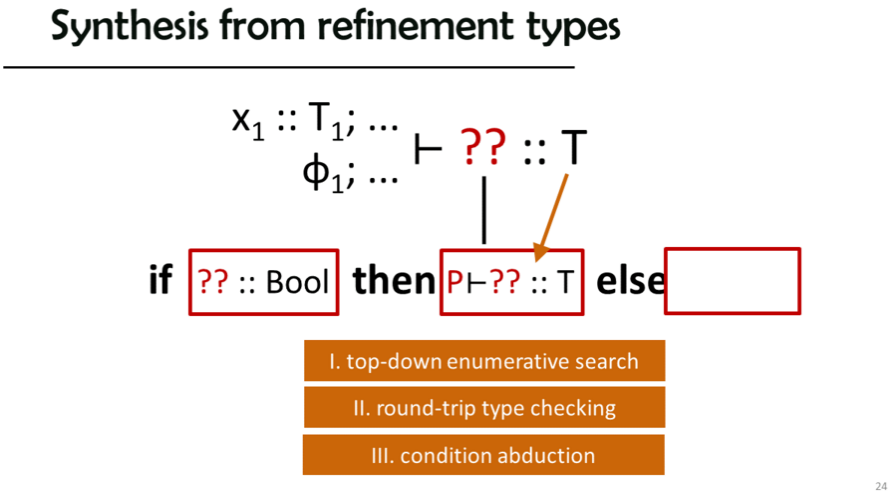
paper [1], at a high-level, though, this form of type checking can impose very tight constraints on each component of the program, dramatically reducing the search space.

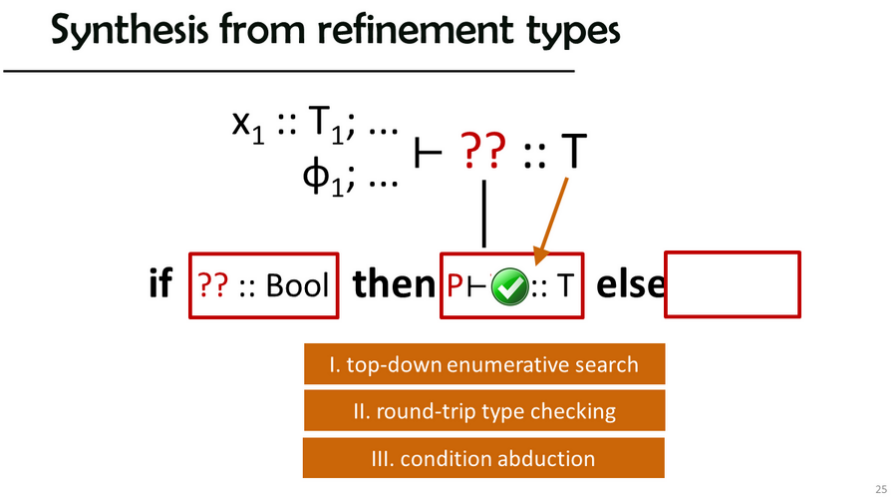
22.2 The Synquid synthesis approach

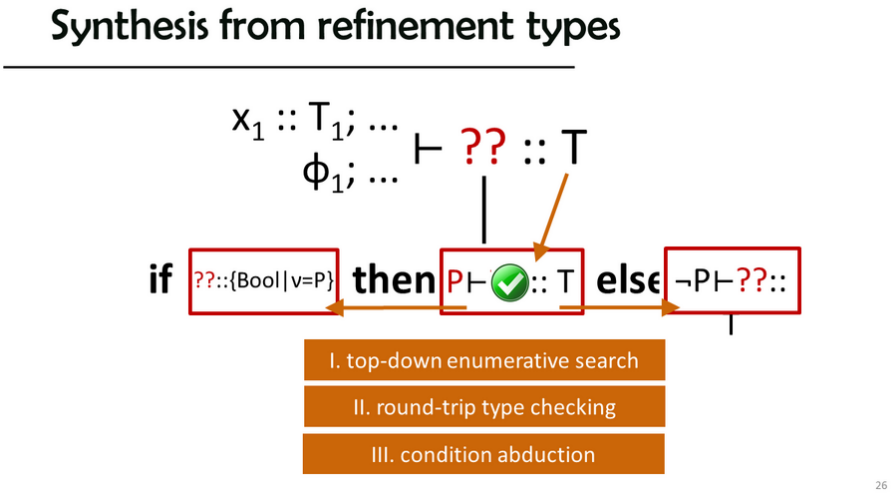
Synquid is a system that implements synthesis via refinement types. There are three key elements to the Synquid synthesis approach.

- Top down search in the style described in Lecture 4.
- Round-trip type checking to prune the space efficiently at each step of the search .
- Condition abduction to generate programs in an incremental way.

22.3 Condition abduction







Condition abduction is a process similar to the STUN approach from Lecture 3. Just like with STUN, the high-level idea is to synthesize a program that works for a subset of the inputs, then generate a program that works for the rest of the inputs, and then combine the two programs together. In this case, though, rather than concrete examples, what we want is the weakest predicate P, such that we can synthesize a program C_t that works assuming P. If P is true, then we are done, if we is false, we can try to synthesize a program C_f that works assuming that P is false. If we can synthesize both of these programs, then we can produce a solution of the form

$$if\ (P)\ C_t\ else\ C_f$$

that is guaranteed to work for all inputs. The process is illustrated by the figure.

22.4 Using Synquid

There is a web interface for Synquid here. The web interface includes a few examples that illustrate some of the key ideas in Synquid. From the examples, you can see that the syntax is basically the ascii version of the more mathematical syntax we have been using in the slides. For example, in place of v, the types in Synquid use `_v`, and the v : is implicit in the type definitions, so for example, the type of $Nat = \{v : Int | v \geq 0\}$ is written in Synquid syntax as `type Nat = {Int | _v >= 0}`.

The examples also illustrate the use of *measures*. A measure is a user defined function that can be used as part of a type, and which has some structural restrictions to ensure that the solver can reason about the types that use these functions. A particularly important class of measures are *termination measures*. These map from a user-defined type to a natural number and are used by the system to ensure that the functions it synthesizes terminate. They basically serve the role of variants or rank functions we saw in Lecture 11. When the synthesizer considers introducing a recursive call, it will only use it if it can prove that some termination measure is decreasing.

References

[1] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program synthesis from polymorphic refinement types. *SIGPLAN Not.*, 51(6):522–538, June 2016. ISSN 0362-1340. doi: 10.1145/2980983.2908093. URL http://doi.acm.org/10.1145/2980983.2908093.